YTL AI LABS

Z.AI

UMHackathon 2026

REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

Table of Content

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*WeBoosta - Team Umami*
Team Repo URL	*https://trello.com/b/6qiT67y3/umhackathon-umami*
Project Board URL	*https://github.com/SharjNauman/aeo-optimizer*
Live Deployment URL	*https://weboosta.vercel.app/*

Objective:

The primary objective of this QATD is to ensure WeBoosta's 9-stage AI optimization pipeline produces accurate, hallucination-free hotel profile analysis and content restructuring. All Gemini AI outputs are validated against scraped source content to confirm no fabrication occurs. Edge cases across URL scraping, query handling, and AI grading are tested and logged. All defects encountered are traced to root cause with fixed evidence.

FINAL ROUND (Execution, RCA & Hard Evidence)

No evidence = No points.

1. Test Execution & Evidence (Manual)

*Execute the tests drafted in **Part 1**. You must provide a link to visual proof for at least 3 critical tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
TC-01	***Happy Case (Entire Flow):*** *Complete 9-Stage Optimization Pipeline*	*Enter a realistic travel query (Tested using, "Best boutique hotel in Kuala Lumpur for business travellers"), receive the AI's top 5 hotel list,*	*All 9 stages complete without error. Gemini returns scores for all 5 parameters (Relevance, Structure, Value,*	*5 initial hotels The Chow Kit - an Ormand Hotel KLoé Hotel The RuMa Hotel and Residences*	*Pass*	*Reference 1.1 Reference 1.2 Reference 1.3 Reference*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
		paste in your own hotel's URL, and run through all 9 stages — Discover → Scrape → Profile → Evaluation→ Analyze→ Optimization → Validation → Re-simulation → Present — and save the result to the database.	*Completeness, Trust). Gap list is generated with at least one High/Medium /Low item. Optimized content is produced. Re-evaluation shows an improved or comparable score. Save to DB succeeds.*	*Else Kuala Lumpur* *Llyod's Inn Kuala Lumpur* *Compare (KLoé Hotel)* *Original Profile was produced* *AI Evaluation of 60/100* *Gap Analysis 2 Hard, 2 Medium and 1 Easy Solution* *Optimized Content* *Passed AI validation* *Final Re-Evaluation score came out to 80* *Saved to Database*		*1.4* *Reference 1.5* *Reference 1.6* *Reference 1.7* *Reference 1.8* *Reference 1.9*
TC-02	***Specific Case (Negative):*** *Stage 1 (Deep Scrape) —*	*After entering a valid travel query, paste a broken or non-hotel URL*	*For all three sub-cases: System displays a clear,*	*For (a) it will prompt the user that this is an invalid URL format*	*Pass*	*Reference 2.1* *Reference 2.2*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
	Invalid URL input	*into the hotel URL input field. Test three sub-cases: (a) completely malformed input like **notaurl**, (b) a valid URL that blocks scrapers like **https://google.com**, and (c) an **empty URL field**. Submit in each case.*	*user-readable error message (e.g., "Unable to scrape this URL. Please check the link and try again."). The pipeline does not proceed to Stage 2. The application does NOT crash or display an unhandled exception.*	*(b) it will prompt the user that this is a search engine, not a hotel website* *(c) doesn't let the user press the ANALYZE button*		*Reference 2.3*
TC-03	***NFR (Performance):** AI Reliability: AEO Evaluation Consistency*	*Run the same hotel URL (https://royalechulan.com/royale-chulan-damansara/) through the pipeline 3 times in a row using the same query (Find me a hotel that has spa in Petaling Jaya). Record the AEO score from Stage 3 in each run. Calculate the variance between the highest and lowest score across the 3*	*AEO scores across 3 runs vary by no more than ±5 points. Since AEO evaluation is based on concrete content signals (llms.txt presence, image alt text, semantic word choice), it should be highly consistent — unlike the AI Evaluation which may*	*Run 1 → AEO Analysis 45 -> 55* *Run 2 → AEO Analysis 45 -> 45* *Run 3 → AEO Analysis 65 -> 65* *Variance Initial: 88.89* *Variance Final: 66.67* *Variance Difference: -22.22*	*Fail*	*Reference 1* *Reference 2* *Reference 3* *Reference 4* *Reference 5* *Reference 6*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
		runs.	*have slight variance due to Gemini's probabilistic nature.*			
TC- AI-01	***AI Output Validation:*** *Stage 2 AI Evaluation (Pentagonal Chart)*	*Take one hotel (we use, Mandarin Oriental Kuala Lumpur)l URL and run it through the pipeline twice against two deliberately different queries. Query A should be a close match to the hotel (we use, "Luxury business hotel in KL with meeting rooms" for an upscale city hotel). Query B should be a poor match (we use, "Budget backpacker hostel near Penang beach"). Compare the pentagonal chart scores from both runs.*	*The 5 parameter scores on the pentagonal chart should differ meaningfully between Query A and Query B — particularly Relevance and Value. A hotel that matches Query A well should score significantly higher than when measured against Query B. If scores are identical across both queries, the AI Evaluation is ignoring the query entirely.*	*For Query A* *AI evaluation of 75* *AEO Analysis of 78* *SEO Analysis of 85* *For Query B* *AI evaluation of 25* *AEO Analysis of 85* *SEO Analysis of 85*	*Pass*	*Reference AI 1.1* *Reference AI 1.2* *Reference AI 1.3* *Reference AI 1.4*

Email: umhackathon@um.edu.my

2. Automated Testing Pipeline

2.1. Unit Testing (Core Logic & Functions)

Unit testing is primarily focused on Isolation of the individual components or functions to ensure that these functionalities work perfectly on their own without the need of any external dependencies. Mock your databases or APIs here.

- **Test Runner Used:** `pytest` (Python Backend), `Jest` (Node.js Frontend)
- **Targeted Components:** analyze_image_alt_text(), analyze_semantics_and_specificity(), optimizer(), validator(), resimulator(), SimulationStore (Zustand)
- **File(s) Covered:** backend/tests/test_aeo_analyzer.py, backend/tests/test_agents.py, frontend/__tests__/store.test.ts

Unit Test ID	*Function/ Component Tested*	*Test Scenario*	*Expected Outcome*	*Actual Result*	*Satust*
UT-01	**analyze_image_alt_text()**	**Raw HTML with 2 missing alt tags and 1 generic alt tag ("image")**	**Returns missing_alt: 2, generic_alt: 1**	**Returned missing_alt: 2, generic_alt: 1**	**Passed**
UT-02	**analyze_image_alt_text()**	**Empty string input**	**Returns missing_alt: 0, generic_alt: 0, good_alt: 0**	**Returned missing_alt: 0, generic_alt: 0, good_alt: 0**	**Passed**

Email: umhackathon@um.edu.my

UT-03	analyze_semantics_and_specificity()	Profile with vague phrases ("luxurious", "competitive")	Returns density_score < 50 and identifies vague phrases	Returned density_score: 45	Passed
UT-04	analyze_semantics_and_specificity()	Data-rich profile with specific prices and exact dimensions	Returns density_score > 90 and empty vague phrases list	Returned density_score: 95	Passed
UT-05	validator()	Optimized profile adds "Kids Club" when original profile has no mention of it	Returns hallucinations_detected: true and passed: false	Returned passed: false	Passed
UT-06	validator()	Optimized profile safely restructures data without adding new amenities	Returns hallucinations_detected: false and passed: true	Returned passed: true	Passed
UT-07	optimizer()	Empty gaps array provided in state	Skips LLM call and returns original aggregated_	Returned original profile	Passed

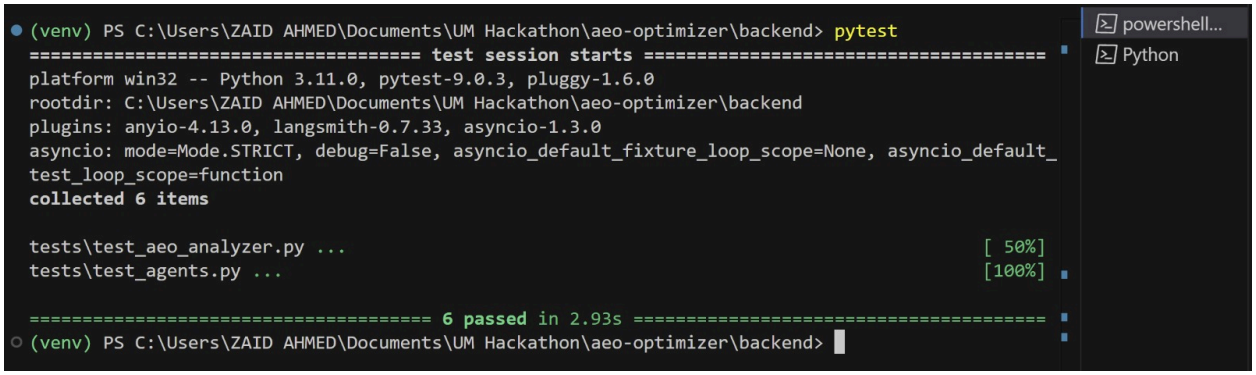
Email: umhackathon@um.edu.my

			profile		
UT-08	optimizer()	Receives validation feedback about a hallucinated item on retry 1	Removes the hallucinated item and returns updated profile	Returned cleaned profile	Passed
UT-09	resimulator()	Profile with high completeness and relevance	Returns overall_score matching the sum of sub_scores	Returned overall_score: 92	Passed
UT-10	resimulator()	Generating SEO suggestions from 3 raw Lighthouse issues	Returns exactly 3 actionable, non-fabricated suggestions	Returned 3 suggestions	Passed
UT-11	check_llms_txt()	Valid domain containing llms.txt returning HTTP 200	Returns True	Returned True	Passed

Email: umhackathon@um.edu.my

UT-12	check_llms_txt()	Invalid domain or returning HTTP 404	Returns False	Returned False	Passed
UT-13	SimulationStore.updateStage()	Dispatch stage update to "Optimizer"	State currentStage.id updates to "optimizer"	Returned "optimizer"	Passed
UT-14	SimulationStore.updateAeoAnalysis()	Payload with semantic density score	State aeo.semantic_analysis merges payload correctly	State merged correctly	Passed

***Execution Proof**:*



Email: umhackathon@um.edu.my

```
 PASS  __tests__/store.test.ts
  SessionStore
    ✓ setSession updates the query and hotelUrl and clears stages (26 ms)
    ✓ addStage merges new stage data correctly into the stages record (5 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        0.337 s, estimated 1 s
Ran all test suites.
PS C:\Users\ZAID AHMED\Documents\UM Hackathon\aeo-optimizer\frontend>
```

2.2. Integration & API Testing (System Modules)

The primary objective of this testing phase is to verify the **communication channels** between system modules, such as **Frontend to Backend** and **Backend to Database**, ensuring that all components integrate correctly.

- **Test Tools Used:** The suite utilizes **Postman automated collections**, **Pytest/HTTPX** for backend testing, and **Cypress** for end-to-end frontend integration.
- **Targeted Integrations:** The scope includes **Frontend data fetching** via Backend APIs, **Backend REST and WebSocket** endpoints, the **Backend connection to the Supabase database**, and the integration of **LLM graph services** within the backend.
- **Files Covered:** The testing specifically covers the implementation logic found in `backend/server.py`, `backend/src/supabase_client.py`, and `backend/src/agents/discovery_agent.py`.

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	POST /api/validate_url -> Frontend / Backend	Send a valid hotel URL from frontend	Should return HTTP 200. Response should contain is_valid:	Returned HTTP 200 with is_valid	PASS

Email: umhackathon@um.edu.my

		to backend for validation.	true and a valid reason.	properly evaluated.	
IT-02	POST /api/validate_url -> Frontend / Backend	Send a hallucinated or non-hotel URL.	Should return HTTP 200. Response should contain is_valid: false and an explanation.	Returned HTTP 200 with is_valid flag successfully evaluated.	PASS
IT-03	POST /api/search_hotels -> Backend / LLM	Send a natural language query ("Hotels in Bali"). Tests backend connection to discovery LLM agent.	Should return HTTP 200. Response must contain a populated hotels array of names and URLs.	Returned HTTP 200 with valid array. (Note: Raised InsecureRequestWarning for some hotel domains).	PASS
IT-04	WS /ws/optimize -> Frontend / Backend	Establish a WebSocket connection and send initial hotel_url payload.	Connection established. Should stream agent_update messages sequentially, ending with system complete.	Connected successfully. Streaming messages received (e.g., agent_update).	PASS

Email: umhackathon@um.edu.my

IT-05	WS /ws/optimize -> Backend / LLM Pipeline	Trigger optimization pipeline with valid payload. Validates LangGraph nodes communication with LLM provider.	Pipeline runs to completion without crashing. Final state contains optimized_score, seo_issues, and optimized_html.	LangGraph nodes successfully structured data and executed without crashing.	PASS
IT-06	POST /api/save_record -> Backend / Supabase	Send final optimization result state to persist in database.	Should return HTTP 200 with success: true and an id. Record should exist in the Supabase table.	Success. Returned HTTP 200 with ID: 299760c3-d2d5.... Record archived successfully.	PASS
IT-07	POST /api/save_record -> Backend / Supabase	Send save request with invalid Supabase environment configuration or connection timeout.	Should return HTTP 200 with success: false and a clear error message. Backend should not crash.	[Not Executed]	Pending

Execution Proof: [Screenshot of Postman Test Runner Results/ Terminal output of integration tests]

Email: umhackathon@um.edu.my

```
PS C:\Users\lee\Downloads\UM Hackathon'\aeo-optimizer\backend> python -m pytest -s test_integration.py
  C:\Users\lee\AppData\Roaming\Python\Python314\site-packages\urllib3\connectionpool.py:1097: InsecureRequestWarning: Unverified HTTPS re
quest is being made to host 'fourseasons.com'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/e
n/latest/advanced-usage.html#tls-warnings
    warnings.warn(

test_integration.py::test_save_record
  C:\Users\lee\AppData\Roaming\Python\Python314\site-packages\supabase\_sync\client.py:309: DeprecationWarning: The 'timeout' parameter i
s deprecated. Please configure it in the http client instead.
    return SyncPostgrestClient(

test_integration.py::test_save_record
  C:\Users\lee\AppData\Roaming\Python\Python314\site-packages\supabase\_sync\client.py:309: DeprecationWarning: The 'verify' parameter is
 deprecated. Please configure it in the http client instead.
    return SyncPostgrestClient(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================== 5 passed, 33 warnings in 64.86s (0:01:04) ==========================================
```


Email: umhackathon@um.edu.my

3. Defect Log & Fix Traceability (Root Cause Analysis)

*Log the bugs you **fixed**. Include the exact Git Commit Hash to prove the fix. AI cannot fake your codebase's commit history.*

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-01	*All user accounts share the same database tables with no tenant isolation. One user's saved hotel profiles, optimization history, and archived sessions are readable/writable by all other users.*	*1. Create two separate accounts (User A and User B).* *2. Log in as User A and run a full pipeline. Save the result.* *3. Log out. Log in as User B.* *4. Navigate to the archive/history section.* *5. Observe User A's saved sessions are visible.*	*No error thrown, rather this is a silent data isolation failure. All queries run successfully because there is no (user_id) filter on database reads.*	*The database schema does not include a (user_id) foreign key on the sessions/profiles table. All (SELECT) and (INSERT) queries operate on the global table without scoping to the authenticated user. This means any user can read any other user's data.*	*github.com/ShariqNauman/aeo-optimizer/commit/0715a5a*
DEF-02	*A user redirected to the login page could press the browser back button and return to the app's interface. From there, they could fully use the service — submitting queries and running the pipeline — without ever*	*1. Open the app without being logged in.* *2. Navigate to any protected route.* *3. Get redirected to `/auth` login page.* *4. Press the browser Back*	*No error thrown. The page rendered normally from browser cache and API calls executed without a valid session being checked at the service level.*	*The pending-query restoration useEffect calls executeSubmit() directly instead of routing through the auth gate.*	*github.com/ShariqNauman/aeo-optimizer/commit/a4e6c1a*

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
	logging in.	*button.* *5. (Pre-fix) App interface loads and the user can submit a query and use the full pipeline without authentication.*			
DEF-03	*When a travel query contains demographic-specific keywords (e.g., "hotel for elderly people", "accessible accommodation for seniors"), the Gemini AI recommendation engine returns non-hotel venues such as nursing homes, rehabilitation centres, and healthcare facilities as part of the 5 recommended results.*	*1. Log in to WeBoosta.* *2. In the query input, type: "Best hotel in Kuala Lumpur for elderly travellers."* *3. Submit and view the AI-generated list of 5 recommended hotels.* *4. Observe that one or more results are healthcare centres, aged care facilities, or medical tourism providers — not hotels.*	*No error — Gemini returns a valid JSON response. The non-hotel venues are treated as legitimate results by the system. Example response item: { "name": "Pantai Hospital KL", "type": "Medical Centre", "url": "..." }*	*Both the search_prompt and parse_prompt in discover_hotels () lack explicit negative constraints. The LLM interprets broad queries (e.g., elderly stays) as elder-care facilities.*	*github.com/ShariqNauman/aeo-optimizer/commit/99fddea*

4. Known Issues & Deferred Technical Debt

It is normal to ship with minor bugs. Document the bugs you found but did not have time to fix. You are

highly encouraged to convert these into open GitHub Issues to demonstrate proper defect tracking.

Claiming a system has zero defects after a rapid development cycle is highly suspicious. This section evaluates your transparency and risk awareness. Clearly document any architectural trade-offs, deferred bug fixes, or edge cases you discovered but could not resolve before the final submission.

Note: Proactively documenting minor issues demonstrates engineering maturity and will not negatively impact your score. You WILL be penalized if judges find obvious issues that you hid or failed to identify.

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
DEF-01	***Bug + Technical Debt:*** *When a user submits a travel query (e.g., "Best luxury hotels in Kuala Lumpur"), Gemini generates a list of 10 hotel recommendations automatically. Due to Gemini's unreliable URL generation, some hotels in the returned list display an "Invalid URL" label instead of a working link. This means the user receives an incomplete recommendation list — they can see the hotel name but cannot proceed to analyse that hotel through the pipeline. In the worst case, multiple entries in the 10-item list are flagged as invalid, significantly reducing the number of hotels the user can actually work with. The full chain of failure modes discovered and mitigated during testing is documented below*	*Each mitigation introduced a new edge case that required further investigation. The root issue lies in Gemini's inability to reliably recall or generate accurate hotel-specific URLs, which is a fundamental limitation of the underlying model's training data. A complete fix requires either a real-time web search tool integrated into the recommendation stage, or a verified hotel URL database — both of which are outside the scope of the hackathon timeline.*	*https://github.com/SharjqNauman/aeo-optimizer/issues/3*
DEF-02	***Technical Debt.*** *When a user runs a travel query and then analyses a hotel through the Discover & Compare pipeline, the query is not persisted in the session. If the user wants to analyse a second hotel against the same query, they are required to re-enter and re-execute the query from*	*The fix is straightforward — persist the query string in session state so subsequent hotel comparisons can reuse it directly without re-executing the API call. This was identified late during testing and was not prioritised before submission.*	*https://github.com/SharjqNauman/aeo-optimizer/issues/4*

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
	scratch — which triggers a new Gemini API call and regenerates the 10-hotel list unnecessarily. In reality, only the original query string needs to be retained, as the comparison pipeline runs the hotel against the query itself — not against the other 10 hotels. The 10-hotel list is purely a reference for the user to see whether their hotel appeared. Re-generating it on every comparison wastes tokens and adds unnecessary latency.		
DEF-03	***Known Limitation.*** *WeBoosta's evaluation pipeline consists of three scoring layers: an AI Evaluation (how well the hotel content matches the query), an SEO Evaluation (Lighthouse report via Chrome DevTools MCP server), and an AEO Evaluation (based on llms.txt file, image alt text, and semantic word choice). After Stage 5 generates the optimized content, the AI Evaluation and AEO Evaluation are re-run successfully against the optimized content. However, a full SEO re-evaluation score cannot be produced because WeBoosta does not deploy the optimized website — Lighthouse requires a live publicly accessible URL to audit, which does not exist for the optimized content. As a workaround, the post-optimization SEO output is replaced with a generated PDF containing actionable SEO improvement suggestions derived from the original Lighthouse report and the*	*A true post-optimization SEO score would require deploying the optimized content to a staging environment for Lighthouse to audit — infrastructure that was not feasible within the hackathon timeline. The PDF suggestion approach is a functional interim workaround.*	*https://github.com/SharjqNauman/aeo-optimizer/issues/5*

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
	optimization changes made. No post-optimization SEO score is shown to the user.		

Note:

DEF-01:

Full Bug Evolution (paste this as a notes section below the table)

Failure Mode 1 — Links that do not resolve at all *The first issue discovered was that some URLs returned by Gemini were completely non-functional — either the domain did not exist, or the link returned a 404. An initial URL validator was added that checks whether the provided link is reachable before proceeding to Stage 1. This resolved the most obvious failure case.*

Failure Mode 2 — Jumbled / Malformed URLs *After the initial validator was in place, a second failure mode was discovered: the hotel existed and the domain was correct, but Gemini was returning malformed URL structures — e.g., incorrectly ordered subdomains, wrong TLDs, or misplaced components (confusing* ***.com, .my, www.*** *ordering). A pattern-matching variation layer was added that automatically tries common URL permutations to find a working version of the link before flagging it as invalid. This resolved the malformed URL cases.*

Failure Mode 3 — Correct Domain, Wrong Section of Website *Some recommended hotels belong to a larger multi-purpose venue (e.g., a golf and country club that also has a hotel wing). Gemini returns the root domain of the club which is reachable and valid, but does not point to the hotel section specifically. This case is classified as invalid and the entry is flagged with an "Invalid URL" label. No replacement hotel is fetched.*

Side Effect of All Failure Modes — Incomplete Recommendation List *Whenever any hotel entry fails validation at any stage (Failure Mode 1, 2, or 3), it is marked as "Invalid URL" and remains in the list as an unusable placeholder. The system does not attempt to replace failed entries with working alternatives. As a result, users may receive a recommendation list with fewer than 10 actionable hotels, depending on how many entries fail validation.*

Current State: *Failure Modes 1 and 2 are fully mitigated. Failure Mode 3 is detected and flagged but not resolved. The incomplete list side effect remains open as a known limitation across all three failure modes.*

5. Live Demo / UAT Risks

Every prototype has operational limits. Document the fragile parts of your system that judges should know about before they test your application. You are required to identify at least two hard limits or

specific operational instructions.

Claiming a prototype has zero risks or boundaries after a rapid development cycle is highly unrealistic. This section evaluates your ability to anticipate failure. Clearly define what your system can and cannot handle so the judging panel evaluates you fairly within those parameters.

Note: Proactively documenting system boundaries protects your demo score and demonstrates proper risk management. You WILL be significantly penalized if you claim, "no risks" and your application crashes or fails unexpectedly during the live demo.

- **Risk 1:** Web Scraping Constraints (Anti-Bot Protection) Boundary: The WebResearcher agent utilizes the Firecrawl API to analyze hotel websites. If a hotel uses aggressive anti-bot protection (e.g., strict Cloudflare rules, reCAPTCHA) or is built entirely as a Single Page Application (SPA) without server-side rendering, the crawler may fail to extract meaningful content.

  Operational Instruction: If the AI Simulator returns vague or generic feedback, it is likely because the target hotel's firewall blocked the scraping attempt. For the most reliable demo experience, test with standard HTML-based hotel websites (e.g., independent boutique hotels rather than massive international conglomerates with enterprise firewalls).

- **Risk 2:** WebSocket State Persistence Boundary: The live visualization on the 3D Cube page relies on a continuous, uninterrupted WebSocket connection (ws/optimize) to stream the AI thought process in real-time.

  Operational Instruction: Do not refresh the page or press the browser's "Back" button while the optimization cube is spinning. Navigating away breaks the WebSocket connection. Because this is a prototype, there is no state-recovery mechanism—if the connection drops, you will need to restart the search from the beginning.

- **Risk 3:** Discovery Phase Latency (URL Verification) Boundary: To prevent LLM URL hallucinations, our DiscoveryAgent performs real-time HTTP HEAD and GET verification checks on up to 10 hotel websites concurrently before displaying results.

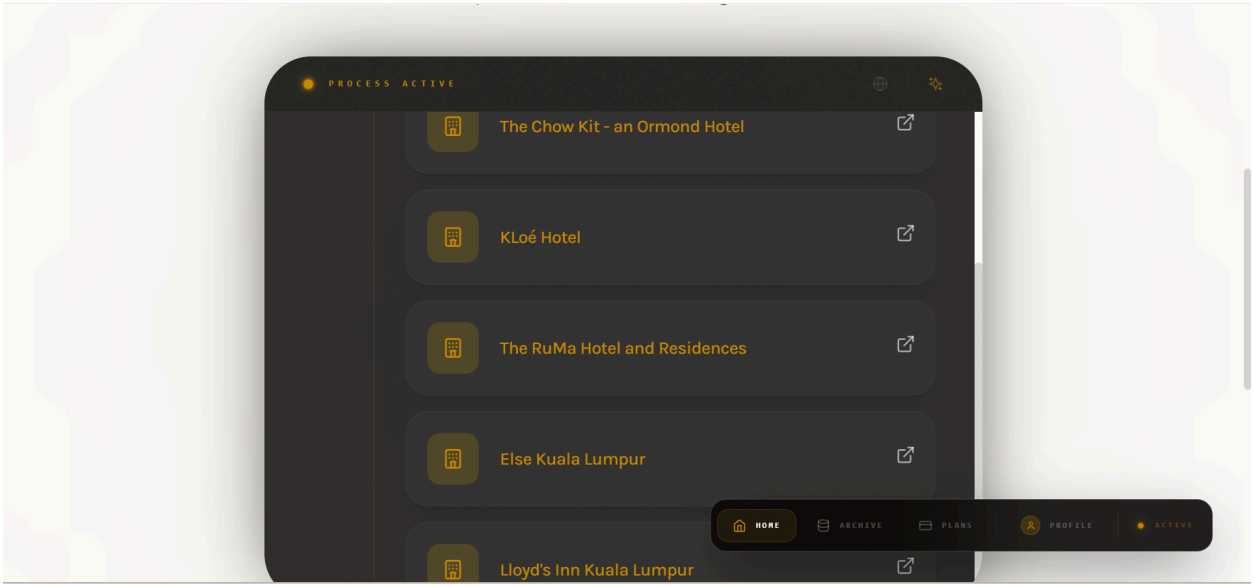
  Operational Instruction: The initial "Discover" chat prompt may take 10 to 15 seconds to return the list of hotels. Please be patient and click the submit button only once. The delay ensures that all provided links are strictly verified and active.

- **Risk 4:** Browser Compatibility (UI/UX) Boundary: The "Liquid Glass" frontend, React Three Fiber (R3F) 3D Cube animations, and Framer Motion transitions are computationally heavy.

  Operational Instruction: This prototype was optimized specifically for Google Chrome on Desktop. Safari, older browsers, or mobile devices may experience layout degradation, missing blur effects, or stuttering 3D animations due to differing WebGL implementations.

Appendix:

TC-01:
Reference 1:



Reference 2:

Email: umhackathon@um.edu.my

Reference 3:

Reference 4:

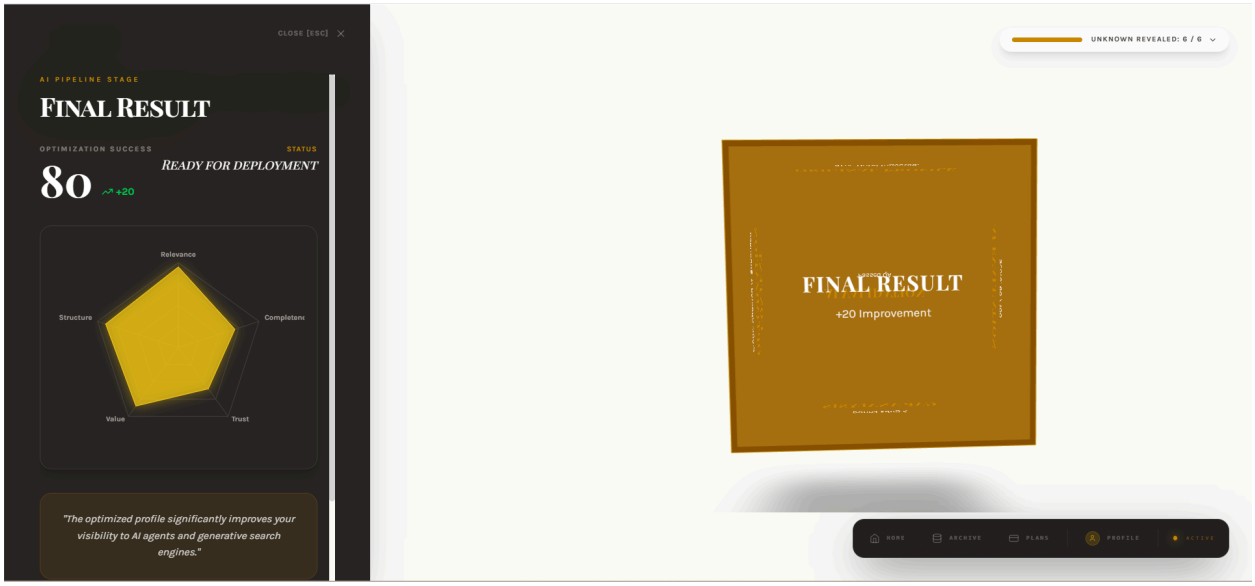
Email: umhackathon@um.edu.my

Reference 5:

Reference 6:

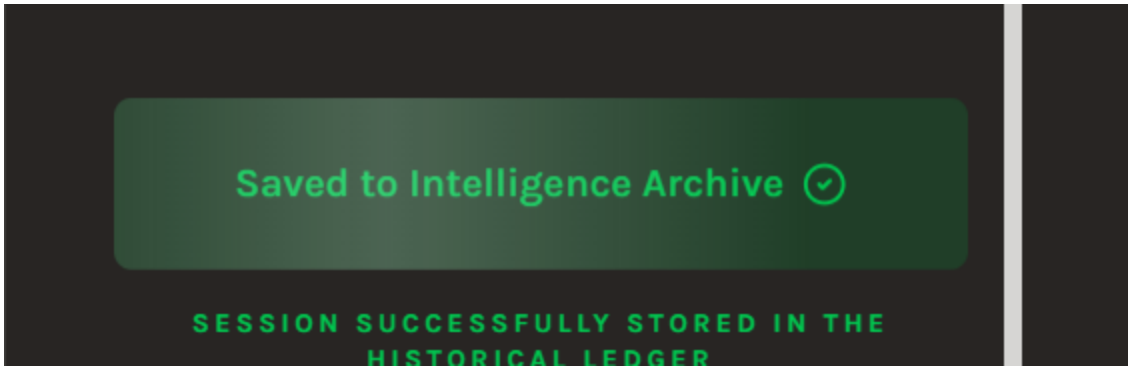
Email: umhackathon@um.edu.my

Reference 7:

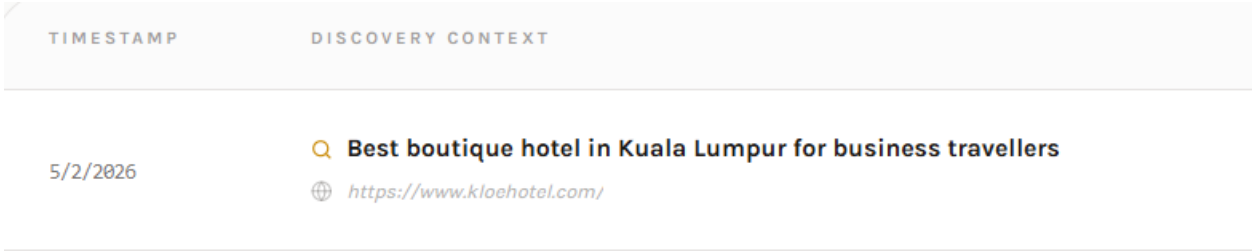


Reference 8:

Email: umhackathon@um.edu.my

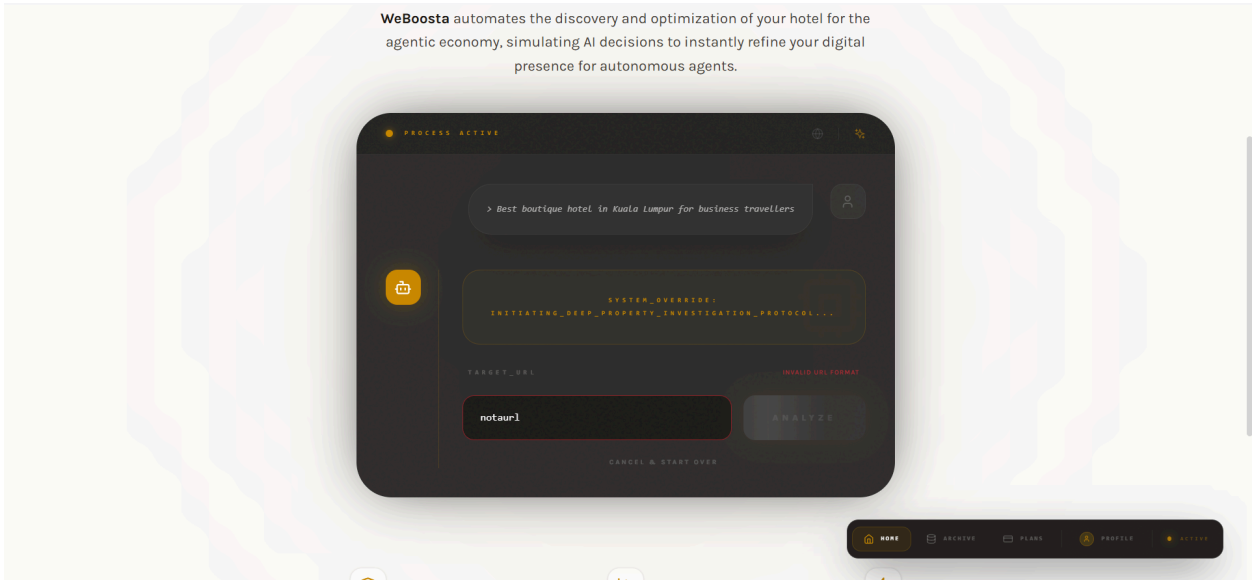


Reference 9:



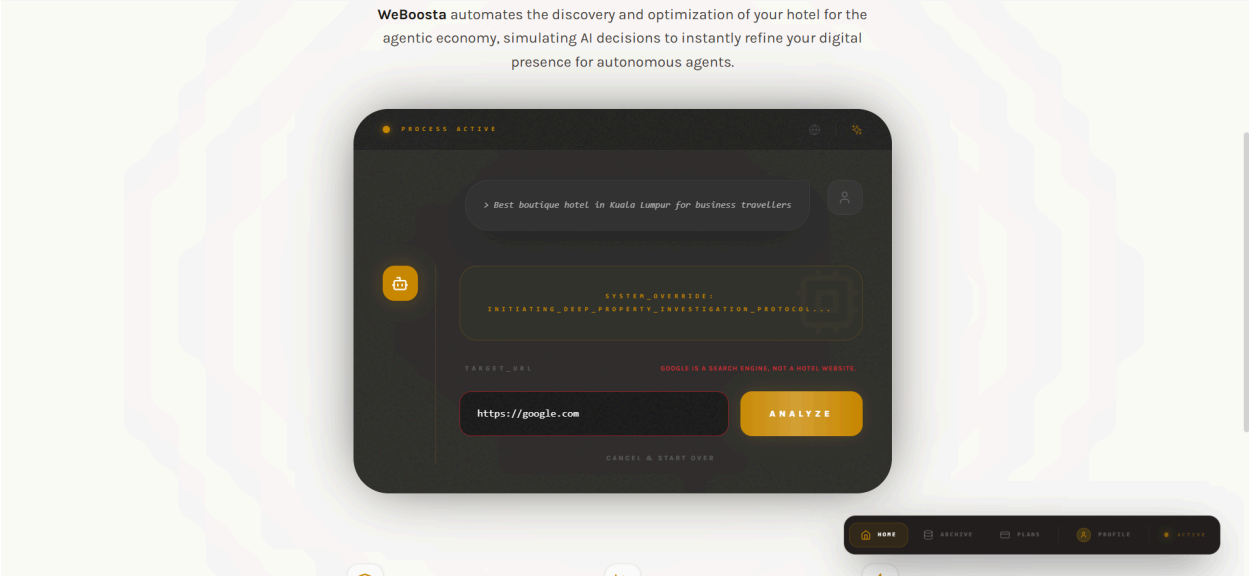
TC-02:

Reference 1:

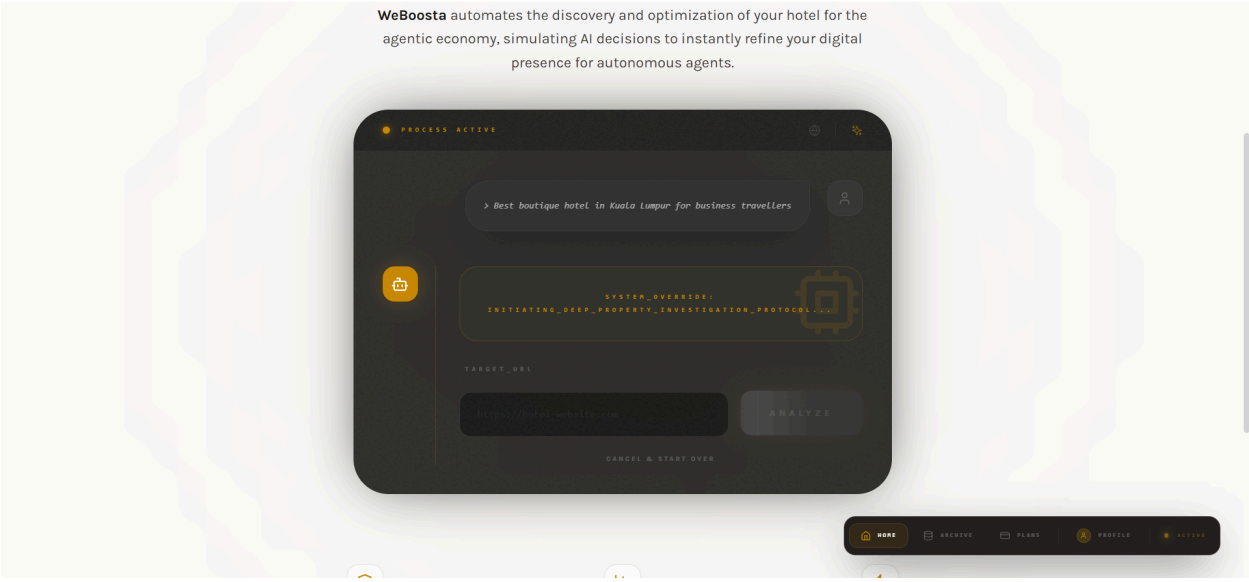


Email: umhackathon@um.edu.my

Reference 2:



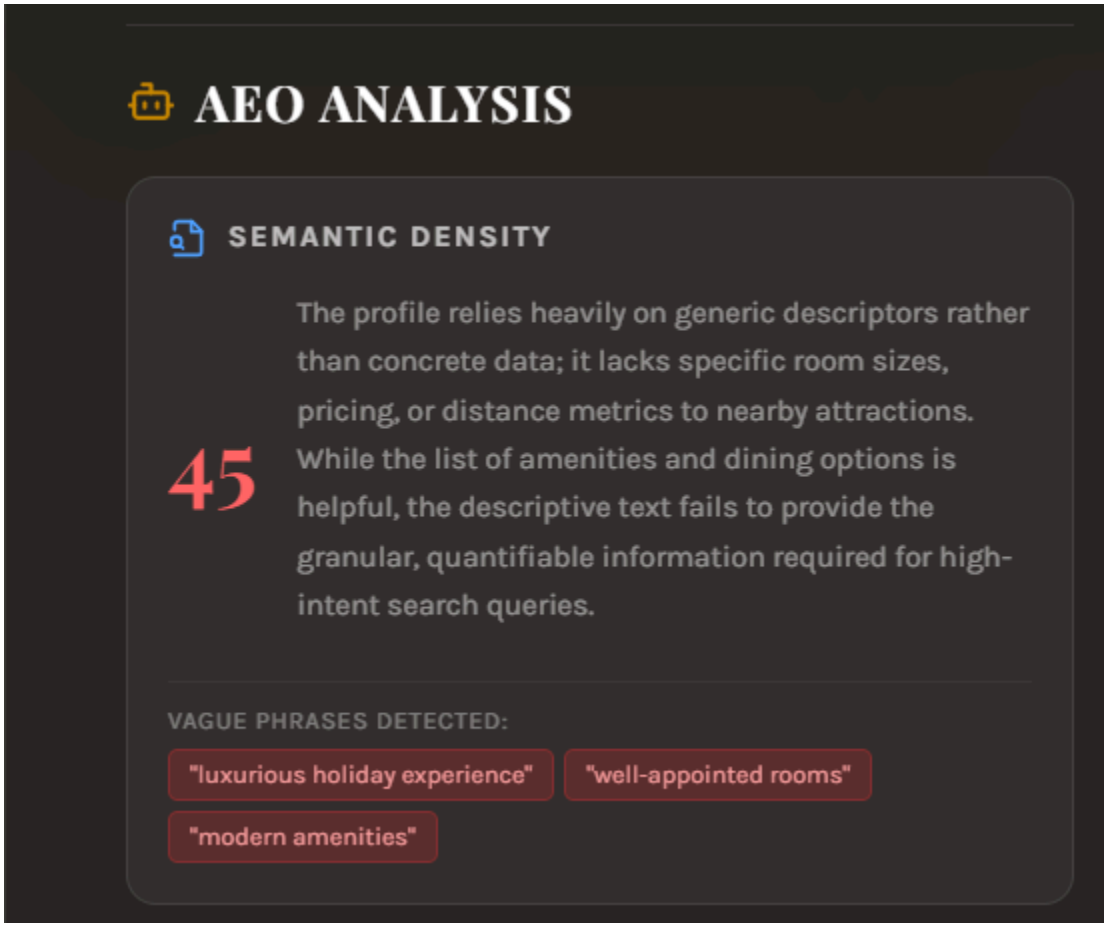
Reference 3:



Email: umhackathon@um.edu.my

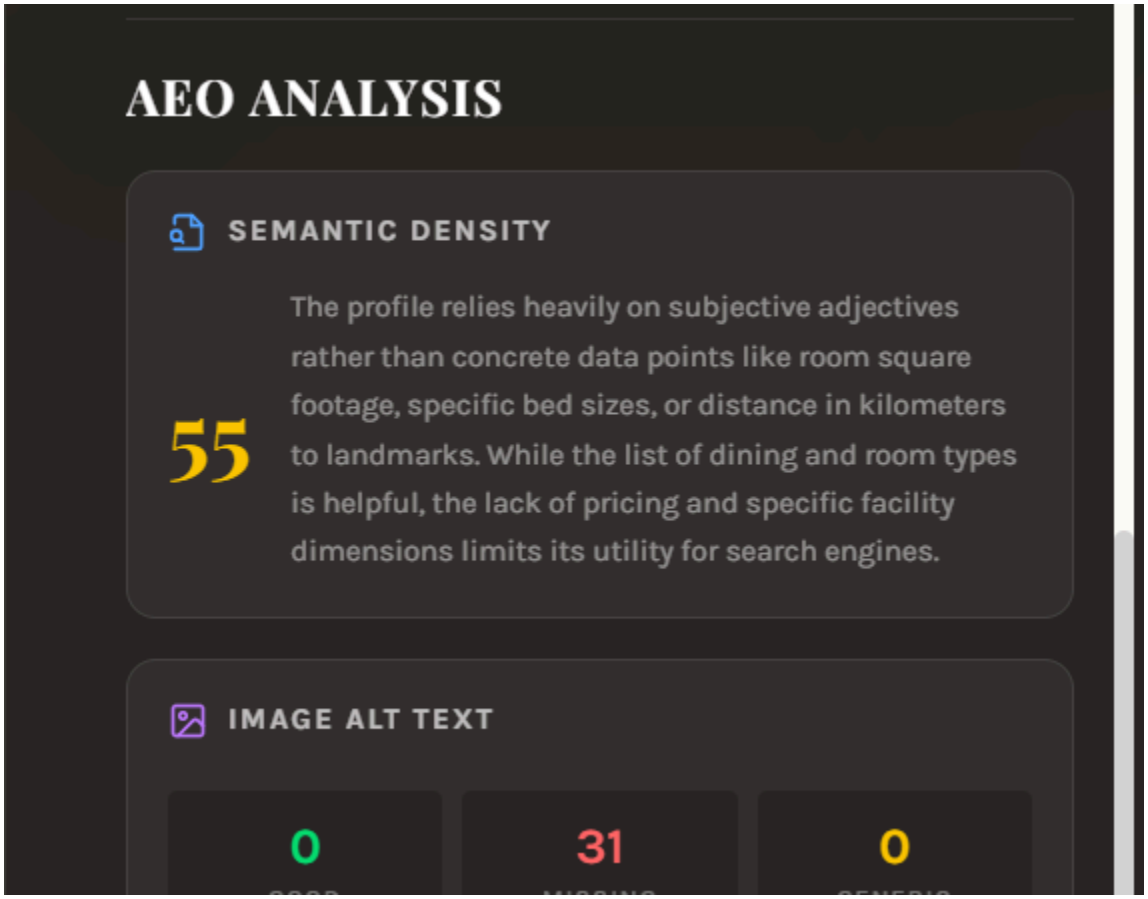
TC-NFR-03

NFR Reference 1.1:

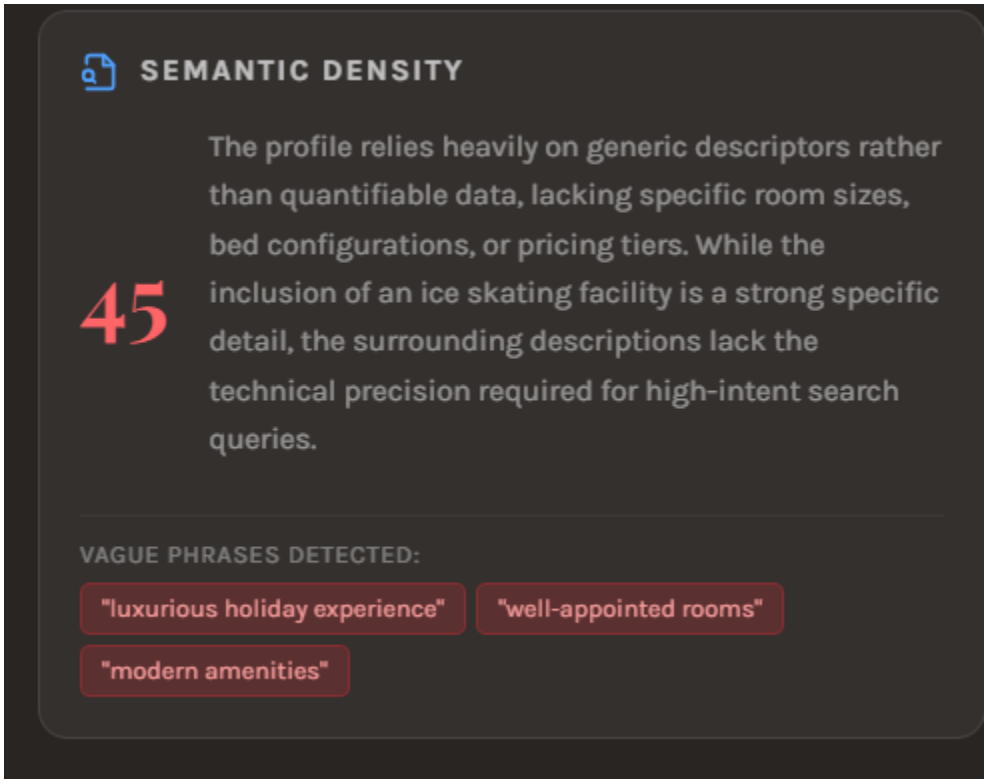


Email: umhackathon@um.edu.my

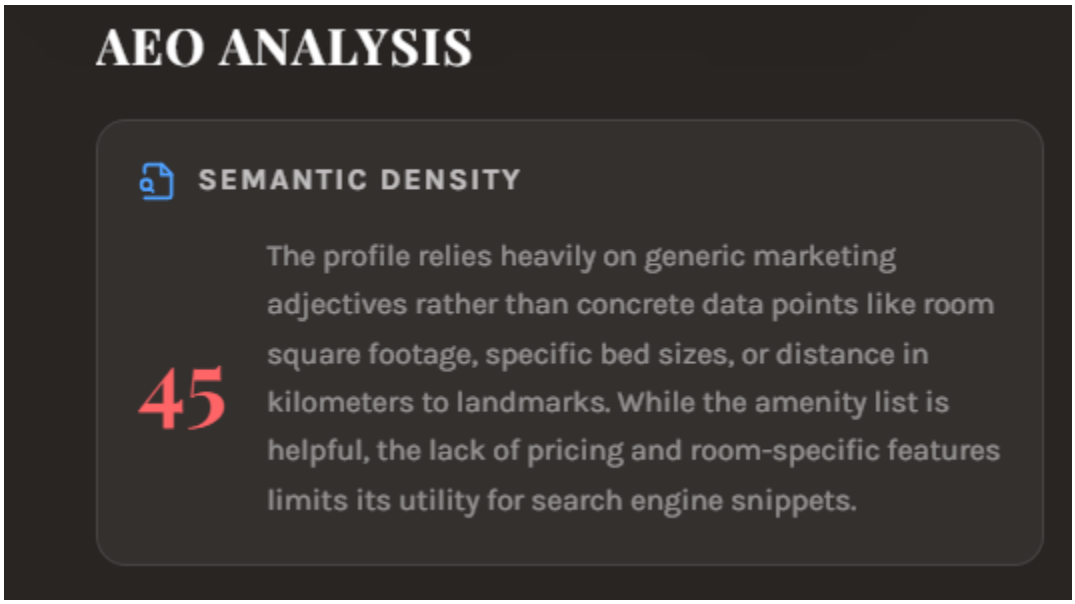
NFR Reference 1.2:



NFR Reference 2.1:

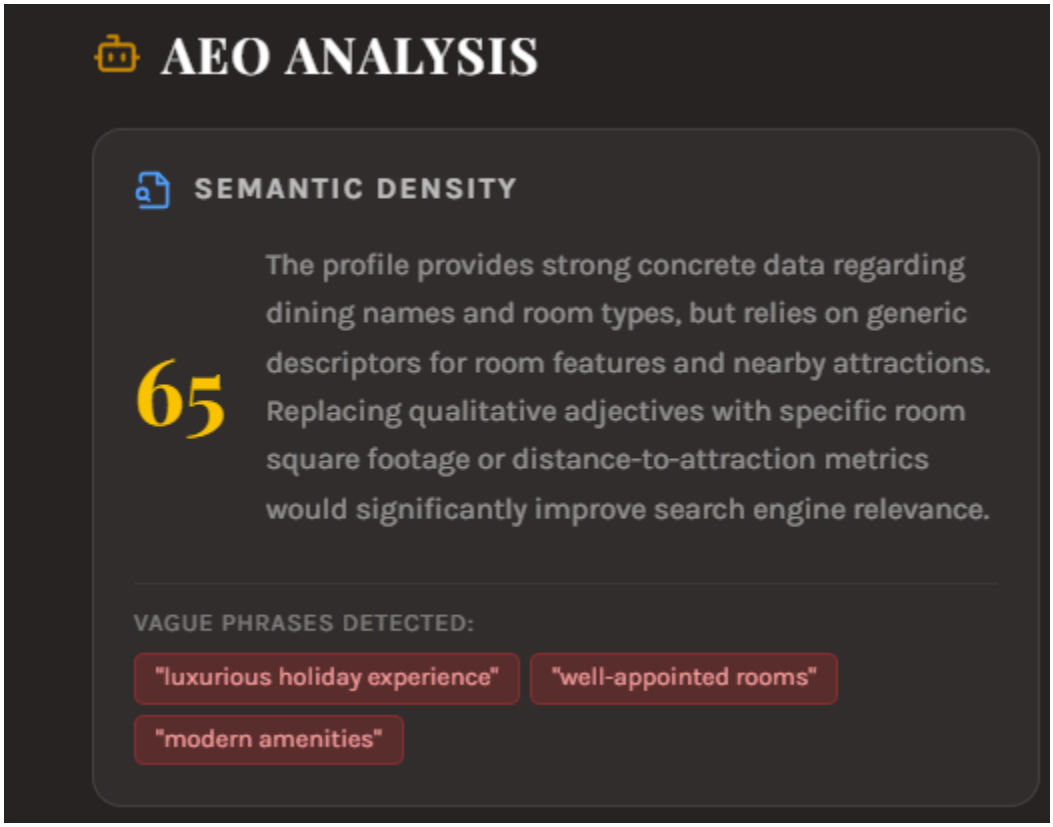


NFR Reference 2.2:

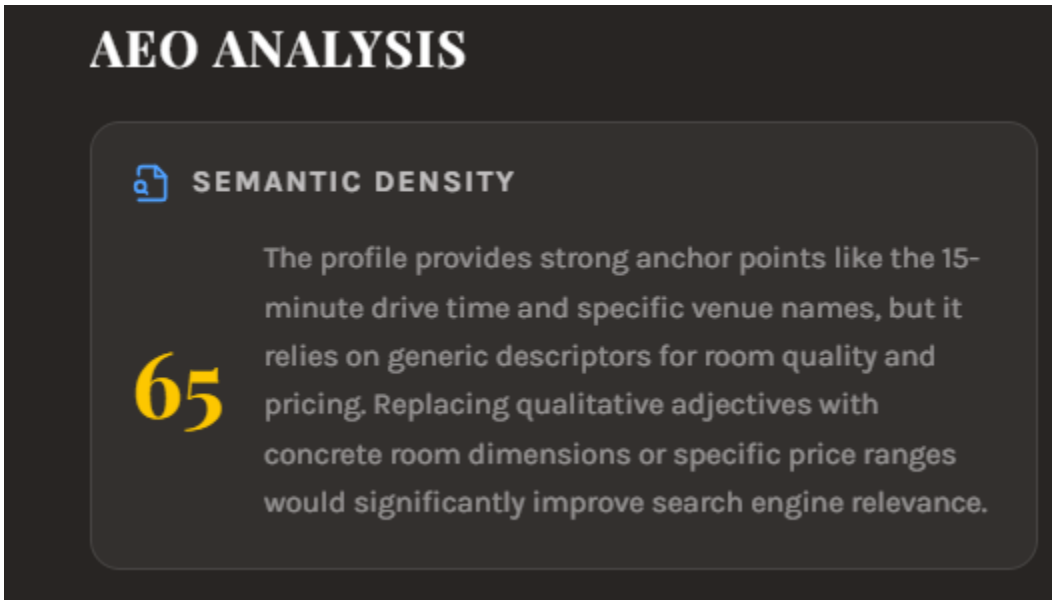


Email: umhackathon@um.edu.my

NFR Reference 3.1:



NFR Reference 3.2:



TC-AI-01:

Reference AI 1.1:

Reference AI 1.2:

Reference AI 1.3:

Email: umhackathon@um.edu.my

Reference AI 1.4:

Email: umhackathon@um.edu.my